

TESTGPT-SERVER: Automatically Testing Microservices with Large Language Models at ByteDance

Jue Wang
wangjue.28@bytedance.com
ByteDance
China

Shuxiang Chen
chenshuxiang.279@bytedance.com
ByteDance
China

Yu Liu
liuyu.kid@bytedance.com
ByteDance
China

Yuan Deng
dengyuan.123@bytedance.com
ByteDance
China

Lei Zhang
zhanglei.dorian@bytedance.com
ByteDance
China

Yuanchang Fu
fuyuanchang@bytedance.com
ByteDance
China

Bo Liu
liubo.666@bytedance.com
ByteDance
China

ABSTRACT

Despite the advantages of the microservice architecture, there is an urgent need for automated testing microservice APIs, ensuring that each API functions correctly and reliably without exhaustive manual effort. This paper introduces TESTGPT-SERVER, a novel, fully automated testing system for microservice APIs developed and deployed at ByteDance, leveraging the capabilities of large language models (LLMs) to enhance test request generation and bug detection. TESTGPT-SERVER utilizes an LLM-enhanced interface analyzer and a code analyzer to extract syntactic and semantic information of the API under test. Based on the extracted information, it employs a targeted request generator and a guided stochastic fuzzer to produce comprehensive test requests. After sending generated test requests to the API, an LLM-assisted response analyzer identifies potential service crashes, and an assertion miner automatically mines functional assertions to detect functional bugs. Deployed at ByteDance, the evaluation results demonstrate that TESTGPT-SERVER is effective, achieving a 40.52% improvement over existing test requests and identifying 50 crash bugs and 2 functional bugs across 14,366 microservice APIs, with minimal manual effort.

ACM Reference Format:

Jue Wang, Shuxiang Chen, Yu Liu, Yuan Deng, Lei Zhang, Yuanchang Fu, and Bo Liu. 2025. TESTGPT-SERVER: Automatically Testing Microservices with Large Language Models at ByteDance. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25)*, June 23–28, 2025, Trondheim, Norway. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3696630.3728545>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

FSE Companion '25, June 23–28, 2025, Trondheim, Norway

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1276-0/2025/06

<https://doi.org/10.1145/3696630.3728545>

1 INTRODUCTION

To manage its large enterprise systems and facilitate reliable service delivery, ByteDance has adopted the microservice architecture [2], which splits the system into thousands of connected web service APIs, including RESTful APIs [22] and RPC-based APIs [41].

Despite the advantages of the microservice architecture, ensuring the correctness of APIs remains a critical challenge [31, 48, 50]. The current approach to microservice API testing in industry heavily relies on manually crafted API-invoking test requests and assertions on the responses [50–52], which is labor-intensive and prone to human error, especially given the sheer number of APIs and the complexity of their logic. This underscores the urgent need for automated testing solutions to efficiently handle the complexity and volume of microservices, ensuring that each API functions correctly and reliably without exhaustive manual effort.

Achieving fully automated testing for microservice APIs involves two main challenges. Firstly, generating high-quality test requests that exercise various behaviors of the API under test is challenging due to the structured nature of API requests whose fields have distinct semantic meanings. For instance, an API, which is responsible for retrieving a list of IDs of videos a user viewed in the client app, executes its main logic only if the request contains a valid user ID in one of its query fields. Moreover, a field of a request may syntactically allow for a string-typed value, while semantically requiring it to be an integer indicating a valid timestamp. Existing techniques often fail to generate test requests that adequately cover the diverse behaviors of microservice APIs [31, 49]. Secondly, automatic bug detection poses a significant challenge [48]. While crash-inducing bugs in RESTful APIs are often indicated clearly by their response status codes, RPC-based APIs do not have standardized response status codes. Additionally, an API's code may contain logical errors that do not cause crashes but lead to inconsistencies between the API behavior and the product requirement documents. Detecting such errors requires functional assertions, for which the automatic generation approach is currently lacking.

To tackle these challenges, we propose TESTGPT-SERVER, an LLM-enhanced, fully automated end-to-end testing system for microservice APIs. Our approach's key insight lies in leveraging the capabilities of large language models (LLMs) [53], which can understand API semantics and generate structured data to enhance traditional techniques for automated test request generation and bug detection. As generally hypothesized [25, 39], we assume that LLMs have captured enough knowledge through their training process to infer the semantic meanings of API request and response fields, and can provide guidance on generating high-quality test requests and identifying abnormal responses. Furthermore, we found that meaningful existing traffic, i.e., requests and their corresponding responses, can be presented in product requirement documents, generated by developers during the development, or recorded during internal use scenarios. Such traffic can provide semantic information to help address the above challenges.

Based on the insights, our system first utilizes an LLM-enhanced interface analyzer to automatically extract syntactic and semantic information of the API from the specification files and existing traffic in a pre-processing phase. Moreover, a code analyzer statically parses the API code to obtain control- and data-flow information. Next, extracted interface and code information are used by two complementary approaches to generate high-quality test requests. Specifically, a targeted request generator utilizes a LLM to generate test requests that reach uncovered code branches of the API. Moreover, a guided, stochastic fuzzer further improves the coverage and address boundary cases. The test requests generated by these techniques are sent to the API under test to obtain responses, which are then analyzed by an LLM-assisted response analyzer to identify potential service crash. Moreover, TESTGPT-SERVER mines functional assertions from existing traffic, which are used for identifying functional errors.

We have deployed TESTGPT-SERVER at ByteDance and evaluated its usability and effectiveness on 14,366 microservice APIs from several production lines. The evaluation results are encouraging. It averagely took 0.004 person-days to deploy TESTGPT-SERVER on a microservice API, achieving a saving of over 2,300 person-day manual effort for testing these APIs. Moreover, it achieved a 40.52% improvement over existing test requests and successfully reported 50 crash bugs and 2 functional bugs, all of which were confirmed by the developers. The primary contributions of the paper are:

- We proposed an LLM-enhanced approach that automatically extracts syntactic and semantic information for microservice APIs, facilitating automated test request generation and bug detection.
- We proposed two complementary approaches to generating high-quality, human-readable test requests that thoroughly exercise the behaviors of a microservice API, an LLM-assisted crash bug identifier for RPC-based APIs, and an automated functional assertion miner for all microservice APIs.
- We integrated all proposed approaches into an LLM-enhanced, fully automated end-to-end testing system for microservice APIs named TESTGPT-SERVER, and deployed it at ByteDance. The evaluation results show that TESTGPT-SERVER can achieve promising code coverage and detect real functional and crash bugs while requiring little manual effort. We also summarized learned lessons and open problems from experimental results and user feedback.

2 TESTGPT-SERVER

Figure 1 illustrates the workflow of TESTGPT-SERVER. It takes the specification description files of the API under test, existing traffic (i.e., request and response pairs), and the code of the API as input. Three phases, namely *pre-processing*, *test request generation*, and *bug detection*, are conducted sequentially. All three phases interact with a large language model for assistance. In the pre-processing phase (discussed in Section 2.1), the interface analyzer analyzes API specifications and existing traffic to extract syntactic and semantic interface information of the API under test, while the code analyzer statically analyzes the API code to extract control- and data-flow information. The extracted information and the initial input are used as inputs for the test request generation phase (detailed in Section 2.2), in which existing traffic is minimized, and used by two complementary components, namely a targeted request generator and a guided, stochastic fuzzer, to generate test requests. The test requests are sent to the deployed API server to obtain responses, which are analyzed in the bug section phase (detailed in Section 2.3) to detect functional and crash bugs. Specifically, the automated bug detector employs an LLM-assisted response analyzer to identify potential service panic. Moreover, a functional assertion miner mines functional assertions from existing traffic to identify functional errors.

2.1 Pre-processing

2.1.1 Analyzing API Interface Syntax and Semantics. The primary objective of the interface analyzer is to accurately determine the syntactic and semantic types of each field ϕ within an API request or response and obtain related information by analyzing the API specification files and existing traffic.

A microservice API typically receives a network request *req* and returns a response *resp*, both structured as trees. Each Node in the tree represents a field ϕ , with leaf nodes representing primitive-type fields and intermediate nodes representing lists, objects, or maps. Each field ϕ is identified by a key $\phi.key$. The difference between an object and a map is that the child fields of an object have a predefined set of keys explicitly defined by developers, while the child fields of a map can have arbitrary keys. For each field ϕ , a unique path $\phi.path$ is constructed by concatenating the field keys from the root to the respective node. For RESTful APIs, a request typically includes three object-type fields under the root: a header (containing meta fields related to authentication and data format, etc.), a query (fields for data retrieval in GET requests), and a body (payload data for PUT or POST operations). RPC-based APIs generally include only headers and body fields in requests. Both types of APIs share a similar response structure, consisting of a status code, a header, and a body containing the payload.

The interface analyzer first extracts syntactic information from the specification files and existing traffic, and then conducts semantic analyses on each field.

Syntax Parsing Based on Specification Files. At ByteDance, microservice API specifications are often declared using the Apache Thrift's Interface Description Language [1] or the Protocol Buffers' proto format [15]. Both formats describe requests and responses in a tree structure. By traversing each tree node from the root, we derive the path for each field. For each field ϕ , the specification file provides

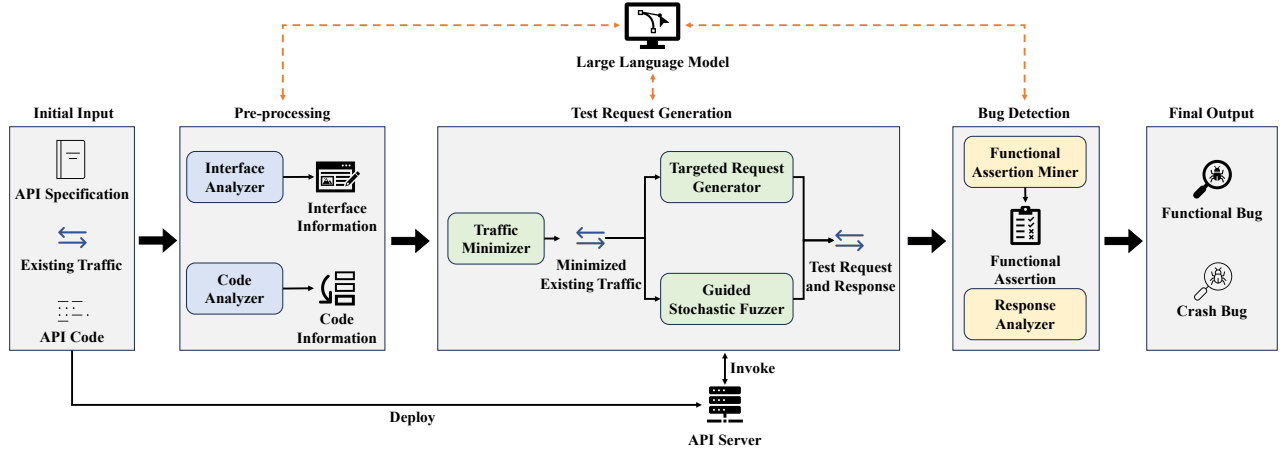


Figure 1: Workflow of the TESTGPT-SERVER

System Prompt: You are an API testing expert that is proficient in the microservice APIs and their analyses.

User Prompt: The content below includes the key and description of a request/response field of a microservice API.

<#Key>: <#Description>

Your task is to determine whether it is of the semantic type <#Type>. You should only output Yes or No.

(a) Prompt template for semantic type identification

System Prompt: You are a code understanding robot that can master the <#Language>. You need to understand the logic of this API interface and find the key field and values that directly affect the logical flow and data processing of this API. Input: Field structure. Output: Key fields and key values, for example: {"AppName": "a", "OsType": "b"}

User Prompt:

This is an example:

Request fields: <#Request Field Information>

Code: <#Example Code>

Output: <#Example Output>

Now please answer:

Request fields: <# Request Field Information>

Code: <# Code>

Output:

(b) Prompt template for key field identification

System Prompt: You are an API testing expert that is proficient in the microservice APIs and their analyses.

User Prompt: The content below includes a request for invoking the API <#API name>, and the corresponding response body.

<#Request>

<#Response Body>

Your task is to determine whether a crash occurs and the service panic. Your answer should start with a simple Yes or No, followed by some explanations.

(d) Prompt template for crash identification

System Prompt: You are a code understanding robot that is proficient in

<#language> and its data analysis, having the highest talent in understanding the <#language> and data analysis.

User Prompt: The following code includes the specific implementation of the interface function, as well as the specific implementation of other functions referenced in this function. In each function implementation, the line number of that line is given at the start of each line, starting from the number 0. The following texts are all the specific implementations of these functions.

<#Request Field Information>

<#API Code>

Your task is to generate a complete specific input parameter for this interface based on the code line I provide. This specific input parameter enables the program to execute up to the specified code line. Specified code line: The code line where <#Line> is located in the <#Function> function.

I will provide an example. You need to generate a request that meets the requirements based on the provided example.

<#Example Request>

Please output the result according to the given example format above. If the parameters in the examples meet the requirements of the task, you can select the value examples; otherwise, please generate parameters that meet the requirements of the task.

Please provide a request that can enable the program to execute to the specified code line: The code line where <#Line> is located in the <#Function> function.

Please directly give the specific values of the request, and do not output anything else. The parameters in the request cannot all be empty.

(c) Prompt template for targeted request generation

Figure 2: Prompt templates used by TESTGPT-SERVER

its syntactic data type $\phi.synType$, whether it is optional $\phi.opt$, and optionally a description in natural language $\phi.desp$. We record these information as the fields's syntactic information. For object fields, we also document the keys of their child fields $\phi.c_keys$.

Syntax Parsing Based on Existing Traffic. In practice, fields not declared in specification files may exist in existing traffic due to frequent interface changes, omissions by developers, or debugging purposes. Therefore, we also analyze existing traffic T , where each

$t \in T$ is a pair of request req and its corresponding response $resp$, to extract all encountered fields, recording their paths, syntactic types, and whether they are optional. Specifically, whether a field is optional is determined by checking the presence of the field across all traffic samples. The parsing results from traffic analysis are integrated with those from specification files, prioritizing traffic-derived syntactic types and specification-derived optionality.

Semantic Analyses of API fields. Primitive fields may have syntactic types such as int, float, string, or boolean. Beyond this, we define additional semantic types based on common API communication patterns:

- **Enumeration:** a field with values from a small, predefined set, indicating specific data types or behaviors.
- **ID:** a field representing a user or product identifier.
- **Date:** a field representing a human-readable date.
- **Timestamp:** a field representing a timestamp, typically in seconds, milliseconds, or microseconds.
- **JSON:** a string field representing a parsable JSON [40] string.

We use the key $\phi.key$, the description $\phi.desp$, and the values $\phi.vals$ that occurred in existing traffic to determine the semantic type $\phi.semType$ of a non-boolean primitive field ϕ with the help of a LLM. Figure 2(a) presents the prompt template for querying the LLM. We identify semantics types as follows:

- **Enumeration.** The specification file may clearly define a field to be an Enumeration field with possible values. Otherwise, we query the LLM for identification with its key and description. If the LLM returns a positive answer, we record all occurred values in traffic as the possible values $\phi.vals$.
- **ID.** We query the LLM for identification with its key and description.
- **Date:** We query the LLM for identification. If the answer is positive, we try to infer and store the date format $\phi.form$ by matching each occurred value against common patterns.
- **Timestamp:** We first try to transform each occurred value to a reasonable date. If all transformation succeeds, we query the LLM of identification.
- **JSON:** We try to parse each occurred value of the field to a Json object and consider it a Json field if all parsing succeeds.

A field ϕ may meet multiple semantic criteria. We prioritize the types according to the order above.

Furthermore, for a string field that is not of the above semantic types, we further determine whether it always semantically represents a number (int or float) or a boolean. If so, we consider it as the semantic type of the field. Finally, For each field ϕ , we analyze its occurred values to record whether it can be null in $\phi.null$ and the T most frequent values in $\phi.freq$ as general semantic information.

Handling Map and List Fields. Map fields, whose child fields can have arbitrary keys, can lead to an explosion in the number of parsed fields based on existing traffic, especially with nested maps. Similarly, list fields, with index-based child field keys, can also cause field explosion. We observe that the child fields of a map or a list normally have the same syntactic and semantic type. Therefore, we identify map fields and list fields in existing traffic and replace their child field keys with a unified key, treating them as identical fields. This process is straightforward for map fields declared in the specification files but requires type inferences for undeclared ones. Empirically, we consider a non-list, non-primitive type field with more than 200 distinct child field keys in existing traffic as a map field; otherwise, it is an object field. For nested maps, iterative identification and key replacement are employed until no new map fields are detected.

2.1.2 Analyzing API Code. The code analyzer performs static control- and data-flow analysis on the API code to extract relations between request fields and code branches. Firstly, the code analyzer constructs a call graph $G = \langle F, E \rangle$ starting the entry point of the API under test, where each node $f \in F$ represents a function and each directed edge $e \in E$ represents a call. This call graph G serves as a foundational structure, mapping out the relationships and interactions between various functions within the API. Secondly, the analyzer conducts an intra-procedural control-flow analysis to identify all conditional branch statements S_f within each method $f \in F$. For each conditional statement $s \in S_f$, the analyzer extracts the set of variable V_s and constants C_s used in the condition's evaluation. Finally, based on obtained information, it conducts an inter-procedural data-flow analysis. Specifically, for each conditional statement s and the corresponding variables V_s , it tries to back-propagate from s to the API's entry point, and identify a set of request fields Φ_v for each $v \in V_s$ such that each field $\phi \in \Phi_v$ may affect the value of v , which in turn can affect the evaluation of the condition of s .

We adopted off-the-shelf static program analysis tools developed within ByteDance to conduct above analyses. For each function $f \in F$, we store its control-flow graph $f.g$, its conditional branch statements $f.S$, the set of used variables $f.V_s$ for each statement $s \in f.S$, and the set of related request fields $f.\Phi_v$ for each $v \in f.V_s$, as attributes of the function in the call graph G .

2.2 Test Request Generation

The test request generation phase takes as input the information extracted by the pre-processing phase and the existing traffic T , outputs test requests, and obtains their corresponding responses.

2.2.1 Traffic Minimization. Existing traffic can be large in amount, especially when it includes recorded real requests and responses, the number of which can reach millions daily. Therefore, TESTGPT-SERVER first reduces the traffic to a reasonable size. Ideally, we can send each request of existing traffic to the API under test to measure its code coverage, retaining only those that uniquely cover some code blocks. However, this approach is time- and resource-consuming, given the large number of requests and low allowed query-per-second of many microservice APIs.

To address this, TESTGPT-SERVER employs a static traffic exploration algorithm to minimize existing traffic. The algorithm takes as input the existing traffic $T = \{\langle req_1, resp_1 \rangle, \dots, \langle req_{|T|}, resp_{|T|} \rangle\}$, syntactic and semantic information of the fields Φ , and code information obtained from code analysis. Firstly, it extracts all fields of primitive syntactic types Φ_0 , and remove those with more than 1,000 distinct values in existing traffic, yielding an subset $\Phi_1 \subseteq \Phi_0$. This is because we observe that fields with excessive distinct values often represent one-time operation IDs or temporary data whose value typically do not affect code coverage and API behavior.

Subsequently, TESTGPT-SERVER identifies all request fields Φ_2 as *key fields* that can affect some conditional branches in the API code based on the static code analysis results. Due to the limitations of static program analysis, Φ_2 might miss some fields, and it is challenging to accurately infer what values of each key field can change the executed code branch. Therefore, we also query a

Algorithm 1: Targeted request generation

```

1 Function TargetedGenerate( $T', \Phi, G = \langle F, E \rangle$ )
2    $Req \leftarrow \emptyset$ ;
3    $B \leftarrow \text{find-targets}(T', G)$ ;
4   for each  $b \in B$  do
5      $b' \leftarrow \text{find-nearest-covered}(G, T')$ ;
6      $Req_{prompt} \leftarrow \text{find-example-request}(T', G, 5)$ ;
7     for  $i \in [1, 5]$  do
8        $req_{prompt} \leftarrow \text{random}(Req_{prompt})$ ;
9        $prompt \leftarrow \text{construct-prompt}(\Phi, G, req_{prompt})$ ;
10       $req \leftarrow \text{query-LLM}(prompt)$ ;
11      if  $req$  reaches  $b$  then
12         $Req \leftarrow Req \cup \{req\}$ ;
13        break
14   return  $Req$ 

```

LLM to identify key fields in request as Φ_3 , and the corresponding key values Val_ϕ for each $\phi \in \Phi_3$ that may execute different code branches. Figure 2(b) presents the prompt template used for the query. Moreover, we dynamically evaluate each field $\phi \in \Phi_1$. Specifically, for each ϕ , we randomly sample 10 requests from T . For each sampled request req , if it contains ϕ , we remove ϕ from req to obtain req' . Otherwise, we add ϕ to obtain req' . We compare the coverage between req and req' to see if ϕ can lead to unique branch coverage. If so, we add ϕ to Φ_2 .

The final key fields can be obtained as $\Phi_\alpha = \Phi_3 \cup (\Phi_2 \cap \Phi_1)$, potentially with key values Val_ϕ for each $\phi \in \Phi_\alpha$. Accordingly, TESTGPT-SERVER extracts a minimal subset of existing traffic $T_M \subseteq T$ that covers as many key fields and key values as possible. Specifically, for each key field $\phi \in \Phi_\alpha$ and each of its corresponding key values $val_\phi \in Val_\phi$, if there exists a $\langle req, resp \rangle \in T$ that $\phi \in req$ and its value is val_ϕ , there must be a $\langle req', resp' \rangle \in T$ that $\phi \in req'$ with its value as val_ϕ . If $|T'|$ is smaller than a threshold, additional traffic is randomly sampled and added to T' as the final minimized traffic.

2.2.2 Targeted Request Generation. Targeted request generation aims to create test requests specifically aimed at covering branches in the API code that have not been reached by existing traffic. We utilize a large language model for the generation.

Algorithm 1 presents the details of the algorithm. It takes as input the minimized set of existing traffic T' , the fields of the API Φ along with extracted syntactic and semantic information, and the code information $G = \langle F, E \rangle$. First, all requests inside T' are sent to the API for coverage measuring, and unreached branches B are identified as targets (Line 3). We query the large language model for each target branch $b \in B$ to generate a test request that reaches b (Lines 4–11).

For querying the large language model, we adapt the few-shot prompting strategy [35]. Specifically, for a target branch b , we find the nearest branch b' that has been reached (Line 5) and use the requests reaching b' as examples (Line 6). The distance between two branches b and b' is defined as the distance to the nearest common upstream node share in the function's control-flow graph. If no other covered branches within the function containing b , we randomly select a covered branch from an upstream function in the call graph. We select at most five requests reaching b' as

Algorithm 2: Guided, Stochastic Fuzzing

```

1 Function Fuzz( $T', \Phi$ )
2    $Pool \leftarrow \emptyset$ ; // request pool
3   for each  $\langle req, resp \rangle \in T'$  do
4      $fitness \leftarrow \text{measure-fit}(req)$ ;
5      $Pool \leftarrow Pool \cup \{\langle req, fitness \rangle\}$ ;
6    $O \leftarrow \text{init-operators}(\Phi)$ ; // mutation operators
7    $P \leftarrow \text{init-porb}(O)$ ; // initial operator selection probability
   distribution
8    $accumulate \leftarrow 0$ ; // accumulated fitness score
9   while not reach time or round limit do
10     $req_p \leftarrow \text{weighted-sample}(Pool)$ ;
11     $o \leftarrow \text{op-select}(O, P)$ ;
12     $req \leftarrow \text{mutate}(req_p, o)$ ;
13     $fitness \leftarrow \text{measure-fit}(req)$ ;
14     $accumulate \leftarrow accumulate + fitness$ ;
15     $Pool \leftarrow Pool \cup \{\langle req, fitness \rangle\}$ ;
16    if have been  $N$  round then
17       $P \leftarrow \text{adjust-prob}(P, accumulate)$ ;
18       $accumulate \leftarrow 0$ ;
19   return  $Pool$ 

```

examples Req_{prompt} . These example requests are chosen because the code paths they execute are highly similar to the paths needed to cover the target branch, thus providing better guidance for the large language model. We construct a prompt using the request fields and their information (including their names, syntactic and semantic types, descriptions, and example values), the API code, the target branch b , and one example requests Req_{prompt} (Line 7), and query the large language model (Line 9) repeatedly. For each time, we randomly select an example request. Figure 2(c) presents the prompt template. We employ a few prompting tricks that are commonly adopted. For instance, we repeatably specify the task, and explicitly define the output format. If the generated test request req successfully reaches b , we store req and move on to the next target (Lines 9–12).

2.2.3 Guided, Stochastic Fuzzing. The guided, stochastic fuzzer aims at further exploring behavior of the API under test, especially how it behaves at boundary or erroneous cases.

Algorithm 2 presents the details of the fuzzing process. The fuzzer takes as input the minimized set of existing traffic T' , and the fields of the API Φ along with their extracted information. The requests of T' are used as the original requests in the request pool, each of which is sent to the API under test to measure its fitness score (Lines 2–5). Next, the fuzzer iterates to generate test requests (Lines 9–18). At each iteration, a request is selected from the request pool as the parent request through weighted random sampling according to the fitness scores (Line 10), and a mutation operator is selected based on the mutation operator scheduling strategy (Line 11). The mutation operator mutates the parent request to generate a new request (Line 12), which is sent to the API to measure its fitness score and added to the request pool (Lines 13–15). Every N rounds, the accumulated fitness score is used to adjust the mutation operator scheduling strategy (Lines 16–18). The iteration continues until the

execution time of the number of generated requests reaches the predefined limit (Line 9).

Mutation Operators. We design mutation operators for request fields of different syntactic and semantic types, which are listed in Table 1. For each field type, we design a few operators to change the value to a legal one to examine how the API under test behaves under normal situations. Moreover, we also design operators to change the value to an illegal one to examine whether the API can correctly handle such erroneous cases. When applying a mutation operator on the parent request, we find all fields that match its target type and randomly select one as its mutation target.

Fitness Score. The fitness score of a request indicates its effectiveness and potential in terms of testing the API. We send the request req to the API and calculate its fitness score $fitness(req)$ based on the coverage score $covScore(req)$ and the response status code score $codeScore(req)$. Particularly, for coverage score, we measure the branch coverage ratio cov , and the number of branches it uniquely covers Δ . The coverage score is accordingly calculated as $covScore(req) = \frac{\Delta}{|B_{uncov}|} + cov$, where B_{uncov} is the set of branches that have not been covered. On the other hand, for the response status code score, we measure how often the status code is encountered. If it has not been encountered, $codeScore(req) = 1$. Otherwise, the score is $codeScore(req) = \frac{1}{c}$, where c is the number of encounters. Accordingly, the fitness score of req is $fitness(req) = covScore(req) + codeScore(req)$.

Mutation Operator Scheduling Strategy. The mutation operation scheduling strategy keeps a probability distribution P over the mutation operators O , which is used to sample a mutation operator for the parent request. During the fuzzing process, we gradually adjust P to yield a probability distribution effective at generating requests with higher fitness scores. To achieve this end, we exploit Gibbs sampling [19] to guide the probability adjusting. The Gibbs sampling is a special case of the Metropolis-Hastings algorithm [14], which is one of the Markov Chain Monte Carlo methods [10] that draw samples from a desired probability distribution, for which direct sampling is difficult.

Intuitively, if we can construct a probability distribution Q based on P , where regions with higher probability densities correspond to P that generate requests with higher fitness scores, we can exploit Gibbs sampling to sample from this region with a higher probability. Given a probability distribution P over the mutation operators O , we follow common methods [20, 42, 43] and set the target probability density function $q(P)$ as $q(P) = \frac{1}{Z} \exp(-\beta * \sum_{req \in Req_P} fitness(req))$ where Z is a normalizing partition function, β a constant which is set as -0.33 following [43], and Req_P is a set of requests generated with P .

We sample from q with Gibbs sampling. We choose Gibbs sampling instead of the standard Metropolis-Hastings algorithm because it is specially designed to sample from a joint distribution of multiple random variables. In our case, we consider each $P(o) \in [0, 1]$ as a variable. Starting from the initial P_0 where $P_0(o) = \frac{1}{|O|}$ for each $o \in O$, we modify each $P(o)$ with a small perturbation while ensuring that $P(o) \in [0, 1]$ and $\sum_{o \in O} P(o) = 1$, yielding a new probability distribution P' . With P' , we generate N requests $Req_{P'}$ and records its accumulated fitness scores $acc(P') = \sum_{req \in Req_{P'}} fitness(req)$. Following the Gibbs sampling algorithm,

the acceptance ratio $accept_ratio(P \rightarrow P') = \min(1, \exp(-\beta * (acc(P) - acc(P'))))$ where P is the previous distribution. Based on the ratio, we decide whether to accept P' or reverse to P .

2.3 Bug Detection

The generated test requests are sent to the API under test to obtain responses. In the bug detection phase, the functional assertion miner mines functional assertions from existing traffic T and the extracted information of fields Φ to detect functional bugs in the API under test, while the response analyzer analyzes the responses of test requests to detect crash bugs. Note that we use existing traffic T to mine functional assertions instead of generated test requests due to that most requests in T exercise correct behaviors of the API under test, facilitating accurate assertion mining.

2.3.1 Functional Assertion Mining. The functional assertions we mined are formalized as propositional formulas [11]. Specifically, a functional assertion α has the following syntax given in the Backus Naur form: $\alpha ::= (p_1 \wedge p_2 \wedge \dots \wedge p_n) \rightarrow q$ where p_i and q are propositions that stand for atomic facts of request-response pairs. Table 2 lists the possible types of propositions in the mined assertion. Intuitively, different value combinations of request fields can exercise different API behaviors (i.e., scenes), indicated by their executed code paths. Thus, we want to mine these value combinations as antecedents, and mine consequents for each antecedent.

Mining Antecedents. An antecedent p of our mined functional assertions can be represented as $\phi_1 = val_1 \wedge \phi_2 = val_2 \wedge \dots \wedge \phi_m = val_m$ where $\phi_i \in \Phi$ is a request field and val_i is a constant value. Such a value combination should execute a unique code path of the API. To identify such combinations, given the code information G , we collect all request fields $\Phi' \subseteq \Phi$ that relate to some variables used in a conditional branch. We randomly sample combinations of request fields of Φ' , analyze the executed paths of requests in existing traffic T to identify values of these fields leading to unique paths, and use them as antecedents.

Mining Consequents. For each mined Antecedent, we collect requests in T that match the antecedent, and mine consequents from the corresponding responses. Particularly, for each proposition type in Table, we evaluate the values of each field in responses and derive a proposition as the consequent if possible. Most proposition types concern only one field, and are straightforward to derive. Others, such as the consistency type, concern more than one field and require evaluating all possible combinations, which may lead to heavy computation if the number of traffic is large. To tackle this, we first sample a small subset of the responses, derive potential consequents, and evaluate them against all responses. If the consequent fails to hold for some responses, we discard it.

2.3.2 Crash Bug Detection. Given the response of a test request, we analyze its status code and the payload body to determine whether a crash occurs and the service panics. For RESTful APIs, their status codes are standardized and a 5XX code normally indicates a service panic. For RPC-based APIs, their status codes are defined by the developers. To tackle this, we ask a large language mode to analyze the payload body of the response to determine whether the service crashes. Figure 2(d) shows the prompt template. If the large

Table 1: Mutation Operators

Operator Name	Target Field Types	Description
Legal child adding operator Illegal child adding operator Child removing operator	Object/List/Map	Add a child field that is the child of this field in some existing traffic Add a child field that is never the child of this field in some existing traffic Remove a child field
Legal Enumeration/ID changing operator Illegal Enumeration/ID changing operator	Enumeration/ID	Change the value of the field to a legal Enumeration/ID value Change the value of the field to an illegal Enumeration/ID value
Past time changing operator Future time changing operator Illegal time changing operator	Date/Timestamp	Change the value of the field to represent a random time in the past Change the value of the field to represent a random time in the future Change the value of the field to one that cannot transform to a date/timestamp
Legal JSON modifying operator Illegal JSON modifying operator	JSON	Change the value of the JSON string that is legal Change the value of the JSON string to an illegal JSON string
In-bound number changing operator Out-bound number changing operator	Int/Float	Change the value of the field within the range observed in existing traffic Change the value of the field outside of the range
Boolean flipping operator	Boolean	Flip the boolean value
String modifying operator	String	Randomly add/delete/modify chars of the string value
Null setting operator	All	change the value of the field to null

Table 2: Proposition Type Template

Type Name	Template	Target Field Type
Enumerate	$v \in \{c_1, c_2, \dots, c_k\}$	All
Not empty	$(v \neq \emptyset) \vee (v \neq \dots)$	All
Range	$v \in [c_1, c_2]$	Int/Float
Non-zero	$v \neq 0$	Int/Float
Numerical compare	$v_1 \text{ rop } v_2$	Int/Float
Arithmetic consistent	$v_1 \text{ op } v_2 = c$	Int/Float
Length range	$\text{len}(v) \in [c_1, c_2]$	String
Starts with	$v.\text{startswith}(c)$	String
Ends with	$v.\text{endswith}(c)$	String
Contain	$v.\text{contains}(c)$	String

Note: v_i is the value of a field; c_i is a constant; *rop* is a relational operator (i.e., $<$, $>$, $\geq$, $\leq$, $=$, $\neq$); *op* is an arithmetic operator (i.e., $+$, $-$, $\times$, $/$); *len* is a function returns the length of the string; *startswith*, *endswith*, and *contains* are functions that determine whether c is the prefix, suffix, sub-string of v , respectively.

language model return a positive answer, we stores its explanation in the report.

3 EVALUATION

We have deployed our automated testing system TESTGPT-SERVER at ByteDance and evaluated its effectiveness with real microservice APIs in production. Our study aims to investigate the following research questions:

- **RQ1:** How difficult is it to deploy TestGPT on microservices in real industrial settings?
- **RQ2:** How effective is TestGPT in terms of code coverage on industrial microservice APIs?
- **RQ3:** How effective is TestGPT in terms of bug finding on industrial microservice APIs?

3.1 Deployment

TESTGPT-SERVER is developed in Golang, Python, and Kotlin based on internal tools and platforms. Currently, our system supports testing microservice APIs written in Golang, since it is the most widely adopted language for APIs at ByteDance. For large language

model, we use the off-the-shelf model Doubao-pro-128k [12], which is developed by ByteDance for general purpose.

Testing microservice APIs with TESTGPT-SERVER requires little manual effort. At ByteDance, microservice APIs are organized using the Product-Service-Module (PSM) architecture. A PSM comprises a set of microservice APIs that share a code base and collectively serve a business objective. To deploy TESTGPT-SERVER on a PSM, the developer needs to write a deployment configuration file specifying the name of the PSM and any environment requirement for running the PSM. We provide a web GUI page for writing such configuration files. Existing Traffic is also provided by developers and stored in Hive tables.

The rest of the testing process for each API of the PSM is independent and fully automatically. Given the configurations, TESTGPT-SERVER Automatically obtain the API specification files of its APIs for the internal ByteDance API Management platform and the code of the PSM from the code repository as initial input. TESTGPT-SERVER automatically compiles the PSM code, instruments the code for measuring coverage, and deploys the PSM and its APIs in a sandbox environment. A test report is produced to the developer when the testing process finishes. The test report includes the generated test requests, code coverage details, mined functional assertions, and detected bugs. For detected crash bugs, TESTGPT-SERVER considers that bugs leading to the same response status code and body as duplicates, and retain only one at random. Similarly, we consider functional bugs violating the same assertions as duplicated. Developers can proceed to diagnose and fix the bug while TESTGPT-SERVER continues testing the APIs. Optionally, developers can provide new traffic and filter out incorrect assertions based on the report.

3.2 Experimental Setup

Subjects. To answer the research questions, we selected microservice APIs of several production lines as our experimental subjects. Specifically, we chose 1,002 PSMs, comprising 14,366 APIs and 1,974,882 lines of code in total. Microservice APIs of 298 PSMs are RESTful APIs, while APIs of the rest 704 PSMs are RPC-based.

Evaluation Metrics and Experimental Setup. To answer RQ1, we deploy TESTGPT-SERVER on the subjects, and measured the person-days required for deployment. Furthermore, we asked the corresponding testing team to evaluate the manual effort in person-days required to conduct manual testing, and assessed the person-days saved by adopting our system. To answer RQ2 and RQ3, we conducted a 24-hour automated testing with TESTGPT-SERVER on each subject. To answer RQ2, we recorded the line coverage achieved by TESTGPT-SERVER on each subject and compared it with coverage of existing testing requests of the API, which are manually created or recorded in real usage scenarios and serve as the baseline. To answer RQ3, we documented bugs reported by TESTGPT-SERVER on these subjects. We manually analyzed these bugs and reported potential real bugs to the developers for confirmation and feedback. Overall, the experiments took $14,366 \text{ APIs} \times 24 \text{ hours} = 344,784$ machine hours, i.e., more than 39 machine years. All experiments were conducted in the testing environment of ByteDance.

3.3 Experimental Results

3.3.1 RQ1-Deployment Effort. During the period from 8th of October 2024 to 10th of December 2024 (53 work days), we corporated with the testing and developing teams of the evaluation subjects to conduct the deployment. TESTGPT-SERVER is successfully deployed on all the 1,002 PSMs and their 14,366 APIs. During the period, we averagely had one person working on the deployment each day, yielding a 53 person-days in total for the deployment. Thus, deploying TESTGPT-SERVER on one PSM averagely takes 0.05 person-day, and 0.004 person-day for an API.

The primary labor costs for deploying TESTGPT-SERVER includes familiarizing and writing the necessary configuration files for deployment, applying for permissions to access the code repositories and run the PSMs in testing environment, creating test accounts to access certain APIs, and addressing issues related to compilation, instrumentation, and running the PSMs. Among these, familiarizing and writing the required configuration files is an essential step, which experienced developers or testers can typically complete efficiently within five minutes for a PSM. On the other hand, compilation and PSM-running issues are less frequently encountered (221 PSMs in our subjects), but they are most time-consuming, often takes 1–2 hours to handle. For instance, conflicts may arise between the code configuration and dependencies of some PSMs and the instrumentation service, which requires code modifications. Additionally, some PSMs require special permission and configurations for running in our sandbox environment. Nonetheless, overall, deploying TESTGPT-SERVER on a microservice PSM is straightforward and requires little manual efforts.

Additionally, the testing teams estimated the required manual efforts for conducting manual testing on the subject based on their experience. On average, a human tester can write testing scripts including test requests and assertions for 4 complex APIs or 8.65 simple APIs each day. For the 14,366 APIs of our subjects, 5,502 APIs are considered complex while 8864 are considered simple. Thus, it requires $5,502 / 4 + 8,864 / 8.65 = 2,400.24$ person-days to conduct manual testing on all APIs. Given that it only takes 53 person-days to deploy TESTGPT-SERVER on these APIs, it demonstrates that

Table 3: Overall coverage results on evaluation subjects. Targeted and Fuzzing refer to our targeted request generation approach and guided fuzzing approach, respectively, and Combined refers to their combination

# LoC	Base		TESTGPT-SERVER		Impr.
	# Cov.	% Cov.	Method	# Cov.	% Cov.
1,974,882	530,454	28.86%	Targeted	642,429	32.53%
			Fuzzing	620,113	31.40%
			Combined	745,321	37.74%

TESTGPT-SERVER can significantly reduce required manual effort for testing.

RQ1: Deploying TESTGPT-SERVER on microservice APIs in real industrial settings is straightforward and requires minimal manual effort. Over a period of 53 workdays, TestGPT was successfully deployed on 1,002 PSMs and their 14,366 APIs, averaging 0.004 person-day per API, while manual testing requires approximately 2,400 person-days.

3.3.2 RQ2-Code Coverage. Table 3 presents the overall coverage results on all evaluation subjects. The 14,366 APIs of our subjects contains 1,974,882 lines of code in total. The existing test requests cover 530,454 lines of code, achieving an 28.86% line coverage.

Combining targeted request generation and guided, stochastic fuzzing, TESTGPT-SERVER cover 745,321 lines of code, achieving an 37.74% line coverage and a 40.52% improvements over existing test requests. We further measured the coverage achieved by the targeted request generator and guided, stochastic fuzzer, respectively. As Table 3 shows, both approaches achieve over 10% improvements over existing test requests. Furthermore, we found that 228,100 lines of code (30.60%) are uniquely covered by either approach, and this demonstrates that the two approaches complements each other. Detailed analysis show that the targeted request generator is effective at generating valid test requests that exercise the common behaviors of the API under test, while the fuzzer shows promises in examining how API handles erroneous cases.

RQ2: TESTGPT-SERVER is demonstrated to be effective in terms of code coverage on industrial microservice APIs. It enhances line coverage from 28.86% to 37.74%, achieving a 40.52% improvement over existing test requests. Moreover, the two proposed test request generation approaches complements each other.

3.3.3 RQ3-Bug Detection. TESTGPT-SERVER reported 52 bugs from test subjects, including 50 crash bugs and 2 functional bugs.

TESTGPT-SERVER reported 50 crash bugs, all of which were confirmed by the developers. Based on the response messages and feedback from developers, we identified the root causes of the bugs, as listed in Table 4. The two most common causes are null pointer dereference (34/50) and index out of range (9/50), which accounts for 86% of the crash bugs. Crash bugs of these types are often triggered by incorrect requests (e.g., a request with an int field whose value is out of range) or requests to retrieval data that does not exist in the database. On the other hand, some crash bugs are caused by incorrect assumptions of requests (5/50). For instance, an API receives an offset and a count from the request, and accordingly retrieves data items whose amount is indicated by the count and position indicated by the offset. The API incorrectly assumes that

Table 4: Bug detection results of TESTGPT-SERVER

No.	Type	Cause	Count	# Targeted	# Fuzzing	# Both
1	Crash	Null pointer dereference	34	10	16	8
2		Index out of range	9	2	3	4
3		Incorrect assumptions	5	5	0	0
4		Missing request fields	2	1	1	0
5	Functional	Code logic error	2	2	0	0
Overall			52	20	20	12

the offset and the count will not be both 0, and crashes otherwise. Finally, some crash bugs are caused by missing request fields (2/50). For instance, an API implicitly requires that the request must contain a `start_date` field. However, it does not specify it as not optional in the specification files, nor does it check whether it exists in the request. Therefore, the API crashes when a request without a `start_date` field is received.

We further categorized reported detected bugs by the approaches that generated the requests triggering the bugs. We found that 12 of the bugs are triggered by both approaches, including 8 null-pointer-dereference bugs and 4 out-of-range bugs. For rest of the bugs, as Table 4 shows, the fuzzer triggered 20 of the crash bugs, 16 of which (80.00%) are caused by null pointer dereference, 3 of which (15.00%) are caused by index out of range, and one of which (5.00%) is caused by missing request fields. On the other hand, the targeted request generator uniquely triggered 18 of the crash bugs, 10 of which are caused by null pointer dereference (55.56%), 5 of which (27.78%) are caused by incorrect assumptions, 2 of which (11.11%) are caused by index out of range, and one of which (5.56%) is caused by missing request fields. As expected, the fuzzer shows promise at generating incorrect test requests exercising boundary behaviors of the API, leading to null pointer dereference or index out of range. On the other hand, the targeted request generator generates requests that should be valid according to the specification files, leading to the exposure of incorrect assumptions made by the developers.

TESTGPT-SERVER also reported 2 functional bugs, both of which were confirmed by the developers. Both of the reported functional bugs were triggered by the targeted test request generator and detected by enumeration assertions. They are both caused by specific code logic errors. For one API of our subjects, its response may contain an invalid message if a request triggering a long-time computation is sent. On the other hand, one API under test returns an incorrect result code when receiving a specific request.

RQ3: *TESTGPT-SERVER is effective in identifying bugs within industrial microservice APIs. It reported 50 crash bugs, all confirmed by developers, with the majority caused by null pointer dereference and index out of range errors. Additionally, TestGPT identified two functional bugs, both confirmed by developers, demonstrating its capability in detecting both crash and functional bugs.*

4 LESSON LEARNED

4.1 Leveraging Large Language Models for Enhanced API Testing

In the TESTGPT-SERVER system, Large Language Models is demonstrated to be effective at automating various tasks of API testing,

including field semantic analysis, key field identification, test request generation, and crash bug detection. For field semantic analysis, traditional methods like template matching often struggle to accurately identify the semantic types of fields due to inconsistent naming conventions. For instance, timestamp fields may be named as `create_time`, `created_at`, or `log_time`. Large Language Models, on the other hand, can precisely determine that they all represent a timestamps. moreover, for test request generation, as analyzed in Section 4.2.2, LLMs exhibit promising capabilities in long code analysis, enabling the generation of test requests that accurately cover target branches. We have also observed that incorporating sample field values and nearest neighbor examples in prompts effectively guides LLMs to understand the role of fields in branch coverage and produce valid test requests.

4.2 Handling Slow APIs

During testing the experimental subjects, we found that some microservice APIs exhibit long response times due to complex logic or limited resources, with response times exceeding 20 seconds in some cases. This latency poses challenges for methods like fuzzing, which rely on rapidly generating numerous test requests. Our targeted request generator can partially address slow-response APIs by generating fewer, high-quality test requests and gathering more information in a single execution. However, more suitable methods are needed to optimize testing for these APIs. For instance, heavy program analysis such as symbolic execution, which requires more time to generate test requests but does not need to invoke the API under test frequently, can be applied on these APIs.

4.3 Generating Semantically Meaningful Test Requests

On the experimental subjects, our automated test request generation approaches achieved desirable code coverage and detected several bugs. However, through discussions with developers, we found that they prioritize test requests that reflect common, real-world use scenarios and uncover bugs in the main business logic code. Automatically generated test requests, such as those from fuzzing, often contain field values lacking real-world semantics, making them difficult for humans to interpret. These boundary test requests can indeed lead to robustness errors in the tested APIs, such as null pointer dereference. Although developers acknowledge these errors, they tend to believe such test requests are unlikely in real-world use scenarios and have limited impact, resulting in less motivation to diagnose and fix these bugs. Moreover, the unintelligibility of these test requests complicates diagnosis. Our targeted request generator can produce test requests with some degree of

real-world semantics, but further constraints on generation direction are needed, which is a focus of our future work.

4.4 Generality of Results

Our proposed methods are also applicable to microservice APIs beyond the experimental subjects. Currently, our system supports testing microservice APIs written in Golang, but the methodology is not restricted to this language, and we plan to extend our implementation to support microservice APIs in other languages. Additionally, our system can test microservice APIs based on different protocols, such as HTTP and RPC. As microservice architecture and network APIs has become the mainstream in modern software development, we believe our methods and the lessons learned from our evaluation provide insights for future directions.

5 RELATED WORK

5.1 Analyzing API Interfaces

The majority of API testing tools perform interface analysis to identify the syntactic structure of requests and responses. Some approaches additionally utilize natural language processing techniques to parse semantic information. NLPtoRest [27] applies traditional natural language processing techniques to analyze the human-readable part of the API specifications to extract example values or field dependencies. Respector [25] proposes to employ static and symbolic program analysis to automatically mine specifications for APIs. RESTInfer [33] utilizes traditional natural language processing techniques to infer constraints between fields from natural language API descriptions. Similarly, D2Spec [46] utilizes traditional machine learning techniques to extract these information. RESTSpecIT [16] dynamically generates requests by a LLM with only the name of the API, and then infers the specifications by analyzing the responses. Different from existing work, TESTGPT-SERVER utilizes LLMs to extract semantic types from specifications and existing traffic, and analyzes API's code for extracting relations between fields and branches.

5.2 Test Request Generation

The majority of API testing tools focuses on generating test requests. Most of the tools are black-box or gray-box. RESTler [9] is the first to introduce black-box stateful RESTful API fuzzing by inferring producer-consumer dependencies from specifications and generate requests based on inferred dependencies and response feedback. RestCt [45] applies combinatorial testing to explore the combinations of fields and field values, while RestTest [38] infers field dependencies and utilizes constraint solvers to generate valid requests. On the other hand, RestTestGen [44] aims at testing APIs in both nominal execution scenarios by generating valid requests and error execution scenarios by generating erroneous ones. The bBoxRT [32] focuses on generating faulty requests in a black-box manner for assessing robustness of the API under test. Godefroid et. al. [21] combines several strategies such as structural schema fuzzing rules and learning data values on-the-fly from responses to generate data payloads embedded in the requests. ExVivoMicroTest [18] monitors the execution of API under test to generate requests for regression tests. Miner [37] focuses on generating long request sequences to

find deep bugs by extending valid sequences incrementally. Moreover, it leverages a neural network model to predict key request fields and values. Meanwhile, Morest [34] dynamically maintains and updates a RESTful-service Property Graph to model the API behaviors and generate request sequences. VOAPI2 [17] leverages the insight that the type of vulnerability in an API is strongly associated with its functionality and generates requests based on API's functionality, which is automatically identified.

While most of the techniques conduct black-box or gray-box testing, EvoMaster [3–5, 8] conducts white-box fuzzing with a series of novel heuristics such as SQL handling [6, 47] and testability transformation [7]. The authors deployed EvoMaster at Meituan, and conducted a series of case studies and improvements in industrial environment [50–52].

In recent years, generating test requests with the aid of LLMs has gained increasing attention. Kim et. al. [30] proposed a multi-agent approach that utilizes four agents to collaborate to conduct test request generation, and a LLM to handle domain-specific value restrictions. They also proposed to incorporate reinforcement learning to prioritize these agents [28]. RESTGPT [29] proposes to generate example field values and rules via LLM to augment the original specifications, which can then be parsed by traditional testing techniques. APITestGenie [39] leverages LLMs to generate test requests directly from business requirements and API specifications. However, it requires developers and testers to provide carefully designed requirement descriptions in the prompt. TESTGPT-SERVER utilizes two complementary approaches to generating high-quality test requests. By combining the power of LLM and traditional fuzzing, it is able to thoroughly examine API's behavior

5.3 Bug Detection for Microservice APIs

Most existing work focuses on detecting crash bugs in RESTful APIs. A few pieces of work aim at detecting functional bugs or security bugs. Schemathesis [24] and QuickREST [26] infer application semantic properties from the API specifications such as required headers or expected content types, and conducts property-based testing on APIs. Luo et. al. [36] proposed to apply metamorphic testing [13] to automatically mine metamorphic relations for detecting logic errors. However, it requires comprehensive empirical studies and experiences to designing proper metamorphic relations for different microservice APIs [13]. Atropos [23] utilizes eight oracles, each covering a common class of vulnerabilities in APIs such as SQL injection and remote code execution, to detect security bugs. TESTGPT-SERVER automatically mines functional assertions from existing traffic while analyzing responses with help from LLM, and thus can detect various bugs.

6 CONCLUSION

We propose a fully automated testing system named TESTGPT-SERVER for microservice APIs, leveraging the capabilities of LLM to enhance test request generation and bug detection. Deployed at ByteDance, the evaluation results demonstrate that TESTGPT-SERVER is effective, achieving a 40.52% improvement over existing test requests and identifying 50 crash bugs and 2 functional bugs across 14,366 microservice APIs, with minimal manual effort.

REFERENCES

- [1] William Abernethy. 2019. *Programmer's Guide to Apache Thrift*. Simon and Schuster.
- [2] Nuha Alshuqayran, Nour Ali, and Roger Evans. 2016. A Systematic Mapping Study in Microservice Architecture. In *2016 IEEE 9th International Conference on Service-Oriented Computing and Applications (SOCA)*. 44–51. <https://doi.org/10.1109/SOCA.2016.15>
- [3] Andrea Arcuri. 2017. Many independent objective (MIO) algorithm for test suite generation. In *Search Based Software Engineering: 9th International Symposium, SSBSE 2017, Paderborn, Germany, September 9–11, 2017, Proceedings 9*. Springer, 3–17.
- [4] Andrea Arcuri. 2017. RESTful API Automated Test Case Generation. In *2017 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. 9–20. <https://doi.org/10.1109/QRS.2017.11>
- [5] Andrea Arcuri. 2019. RESTful API Automated Test Case Generation with EvoMaster. *ACM Trans. Softw. Eng. Methodol.* 28, 1, Article 3 (Jan. 2019), 37 pages. <https://doi.org/10.1145/3293455>
- [6] Andrea Arcuri and Juan P. Galeotti. 2020. Handling SQL Databases in Automated System Test Generation. *ACM Trans. Softw. Eng. Methodol.* 29, 4, Article 22 (July 2020), 31 pages. <https://doi.org/10.1145/3391533>
- [7] Andrea Arcuri and Juan P. Galeotti. 2020. Testability transformations for existing APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. IEEE, 153–163.
- [8] Andrea Arcuri, Man Zhang, and Juan Galeotti. 2024. Advanced White-Box Heuristics for Search-Based Fuzzing of REST APIs. *ACM Trans. Softw. Eng. Methodol.* 33, 6, Article 142 (June 2024), 36 pages. <https://doi.org/10.1145/3652157>
- [9] Vaggelis Atlidakis, Patrice Godefroid, and Marina Polishchuk. 2019. RESTler: Stateful REST API Fuzzing. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 748–758. <https://doi.org/10.1109/ICSE.2019.00083>
- [10] Stephen Brooks. 1998. Markov chain Monte Carlo method and its application. *Journal of the royal statistical society: series D (the Statistician)* 47, 1 (1998), 69–100.
- [11] Hans Kleine Büning and Theodor Lettmann. 1999. *Propositional logic: deduction and algorithms*. Vol. 48. Cambridge University Press.
- [12] ByteDance. [n. d.]. Doubao. <https://www.doubao.com/chat/https://www.doubao.com/chat/>. Accessed: 2025-01-14.
- [13] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, TH Tse, and Zhi Quan Zhou. 2018. Metamorphic testing: A review of challenges and opportunities. *ACM Computing Surveys (CSUR)* 51, 1 (2018), 1–27.
- [14] Siddhartha Chib and Edward Greenberg. 1995. Understanding the metropolis-hastings algorithm. *The american statistician* 49, 4 (1995), 327–335.
- [15] Chris Currier. 2022. Protocol buffers. In *Mobile Forensics—The File Format Handbook: Common File Formats and File Systems Used in Mobile Devices*. Springer, 223–260.
- [16] Alix Decrop, Gilles Perrouin, Mike Papadakis, Xavier Devroey, and Pierre-Yves Schobbens. 2024. You Can REST Now: Automated Specification Inference and Black-Box Testing of RESTful APIs with Large Language Models. *arXiv preprint arXiv:2402.05102* (2024).
- [17] Wenlong Du, Jian Li, Yanhao Wang, Libo Chen, Ruijie Zhao, Junmin Zhu, Zhengguang Han, Yijun Wang, and Zhi Xue. 2024. Vulnerability-oriented Testing for RESTful APIs. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 739–755. <https://www.usenix.org/conference/usenixsecurity24/presentation/du>
- [18] Luca Gazzola, Maayan Goldstein, Leonardo Mariani, Marco Mobilio, Itai Segall, Alessandro Tundo, and Luca Ussi. 2023. ExVivoMicroTest: ExVivo testing of microservices. *Journal of Software: Evolution and Process* 35, 4 (2023), e2452.
- [19] Alan E Gelfand. 2000. Gibbs sampling. *Journal of the American statistical Association* 95, 452 (2000), 1300–1304.
- [20] Walter R Gilks, Sylvia Richardson, and David Spiegelhalter. 1995. *Markov chain Monte Carlo in practice*. CRC press.
- [21] Patrice Godefroid, Bo-Yuan Huang, and Marina Polishchuk. 2020. Intelligent REST API data fuzzing. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Virtual Event, USA) (ESEC/FSE 2020)*. Association for Computing Machinery, New York, NY, USA, 725–736. <https://doi.org/10.1145/3368089.3409719>
- [22] Amid Golmohammadi, Man Zhang, and Andrea Arcuri. 2023. Testing RESTful APIs: A Survey. *ACM Trans. Softw. Eng. Methodol.* 33, 1, Article 27 (Nov. 2023), 41 pages. <https://doi.org/10.1145/3617175>
- [23] Emre Güler, Sergej Schumilo, Moritz Schloegel, Nils Bars, Philipp Götz, Xinyi Xu, Cemal Kaygusuz, and Thorsten Holz. 2024. Atropos: Effective fuzzing of web applications for server-side vulnerabilities. In *USENIX Security Symposium*.
- [24] Zac Hatfield-Dodds and Dmitry Dygalo. 2022. Deriving semantics-aware fuzzers from web API schemas. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 345–346. <https://doi.org/10.1145/3510454.3528637>
- [25] Ruikai Huang, Manish Motwani, Idel Martinez, and Alessandro Orso. 2024. Generating REST API Specifications through Static Analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal) (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 107, 13 pages. <https://doi.org/10.1145/3597503.3639137>
- [26] Stefan Karlsson, Adnan Čaušević, and Daniel Sundmark. 2020. QuickREST: Property-based Test Generation of OpenAPI-Described RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 131–141. <https://doi.org/10.1109/ICST46399.2020.00023>
- [27] Myeongsoo Kim, Davide Corradini, Saurabh Sinha, Alessandro Orso, Michele Pasqua, Rachel Tzoref-Brill, and Mariano Ceccato. 2023. Enhancing REST API Testing with NLP Techniques. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (Seattle, WA, USA) (ISSTA 2023)*. Association for Computing Machinery, New York, NY, USA, 1232–1243. <https://doi.org/10.1145/3597926.3598131>
- [28] Myeongsoo Kim, Saurabh Sinha, and Alessandro Orso. 2023. Adaptive REST API Testing with Reinforcement Learning. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 446–458. <https://doi.org/10.1109/ASE56229.2023.00218>
- [29] Myeongsoo Kim, Tyler Stennett, Dhruv Shah, Saurabh Sinha, and Alessandro Orso. 2024. Leveraging Large Language Models to Improve REST API Testing. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (Lisbon, Portugal) (ICSE-NIER'24)*. Association for Computing Machinery, New York, NY, USA, 37–41. <https://doi.org/10.1145/3639476.3639769>
- [30] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. 2024. A Multi-Agent Approach for REST API Testing with Semantic Graphs and LLM-Driven Inputs. *arXiv preprint arXiv:2411.07098* (2024).
- [31] Myeongsoo Kim, Qi Xin, Saurabh Sinha, and Alessandro Orso. 2022. Automated test generation for REST APIs: no time to rest yet. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (Virtual, South Korea) (ISSTA 2022)*. Association for Computing Machinery, New York, NY, USA, 289–301. <https://doi.org/10.1145/3533767.3534401>
- [32] Nuno Laranjeiro, João Agnelo, and Jorge Bernardino. 2021. A Black Box Tool for Robustness Testing of REST Services. *IEEE Access* 9 (2021), 24738–24754. <https://doi.org/10.1109/ACCESS.2021.3056505>
- [33] Yi Liu. 2022. RESTInfer: automated inferring parameter constraints from natural language RESTful API descriptions. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Singapore, Singapore) (ESEC/FSE 2022)*. Association for Computing Machinery, New York, NY, USA, 1816–1818. <https://doi.org/10.1145/3540250.3559078>
- [34] Yi Liu, Yuekang Li, Gelei Deng, Yang Liu, Ruiyuan Wan, Runchao Wu, Dandan Ji, Shiheng Xu, and Minli Bao. 2022. Morest: model-based RESTful API testing with execution feedback. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 1406–1417. <https://doi.org/10.1145/3510003.3510133>
- [35] Robert L Logan IV, Ivana Balažević, Eric Wallace, Fabio Petroni, Sameer Singh, and Sebastian Riedel. 2021. Cutting down on prompts and parameters: Simple few-shot learning with language models. *arXiv preprint arXiv:2106.13353* (2021).
- [36] Gang Luo, Xi Zheng, Huai Liu, Rongbin Xu, Dinesh Nagumothu, Ranjith Janapareddi, Er Zhuang, and Xiao Liu. 2020. Verification of microservices using metamorphic testing. In *Algorithms and Architectures for Parallel Processing: 19th International Conference, ICA3PP 2019, Melbourne, VIC, Australia, December 9–11, 2019, Proceedings, Part I 19*. Springer, 138–152.
- [37] Chenyang Lyu, Jiacheng Xu, Shouling Ji, Xuhong Zhang, Qinying Wang, Binbin Zhao, Gaoning Pan, Wei Cao, Peng Chen, and Raheem Beyah. 2023. MINER: A Hybrid Data-Driven Approach for REST API Fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 4517–4534. <https://www.usenix.org/conference/usenixsecurity23/presentation/lyu>
- [38] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2020. RESTest: Black-box constraint-based testing of RESTful web APIs. In *Service-Oriented Computing: 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings 18*. Springer, 459–475.
- [39] André Pereira, Bruno Lima, and João Pascoal Faria. 2024. APITestGenie: Automated API Test Generation through Generative AI. *arXiv preprint arXiv:2409.03838* (2024).
- [40] Felipe Pezoa, Juan L Reutter, Fernando Suarez, Martín Ugarte, and Domagoj Vrgoč. 2016. Foundations of JSON schema. In *Proceedings of the 25th international conference on World Wide Web*. 263–273.
- [41] R. Santonico. 2004. RPC understanding and future perspectives. *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 533, 1 (2004), 1–6. <https://doi.org/10.1016/j.nima.2004.06.160> Proceedings of the Seventh International Workshop on Resistive Plate Chambers and Related Detectors.
- [42] Eric Schkufza, Rahul Sharma, and Alex Aiken. 2013. Stochastic superoptimization. *ACM SIGARCH Computer Architecture News* 41, 1 (2013), 305–316.

- [43] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, stochastic model-based GUI testing of Android apps. In *Proceedings of the 2017 11th joint meeting on foundations of software engineering*. 245–256.
- [44] Emanuele Viglianisi, Michael Dallago, and Mariano Ceccato. 2020. RESTTEST-GEN: Automated Black-Box Testing of RESTful APIs. In *2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST)*. 142–152. <https://doi.org/10.1109/ICST46399.2020.00024>
- [45] Huayao Wu, Lixin Xu, Xintao Niu, and Changhai Nie. 2022. Combinatorial testing of RESTful APIs. In *Proceedings of the 44th International Conference on Software Engineering (Pittsburgh, Pennsylvania) (ICSE '22)*. Association for Computing Machinery, New York, NY, USA, 426–437. <https://doi.org/10.1145/3510003.3510151>
- [46] Jinqiu Yang, Erik Wittern, Annie T. T. Ying, Julian Dolby, and Lin Tan. 2018. Towards extracting web API specifications from documentation. In *Proceedings of the 15th International Conference on Mining Software Repositories (Gothenburg, Sweden) (MSR '18)*. Association for Computing Machinery, New York, NY, USA, 454–464. <https://doi.org/10.1145/3196398.3196411>
- [47] Man Zhang and Andrea Arcuri. 2021. Enhancing resource-based test case generation for RESTful APIs with SQL handling. In *International Symposium on Search Based Software Engineering*. Springer, 103–117.
- [48] Man Zhang and Andrea Arcuri. 2023. Open Problems in Fuzzing RESTful APIs: A Comparison of Tools. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 144 (Sept. 2023), 45 pages. <https://doi.org/10.1145/3597205>
- [49] Man Zhang and Andrea Arcuri. 2023. Open Problems in Fuzzing RESTful APIs: A Comparison of Tools. *ACM Trans. Softw. Eng. Methodol.* 32, 6, Article 144 (Sept. 2023), 45 pages. <https://doi.org/10.1145/3597205>
- [50] Man Zhang, Andrea Arcuri, Yonggang Li, Yang Liu, and Kaiming Xue. 2023. White-Box Fuzzing RPC-Based APIs with EvoMaster: An Industrial Case Study. *ACM Trans. Softw. Eng. Methodol.* 32, 5, Article 122 (July 2023), 38 pages. <https://doi.org/10.1145/3585009>
- [51] Man Zhang, Andrea Arcuri, Yonggang Li, Kaiming Xue, Zhao Wang, Jian Huo, and Weiwei Huang. 2022. Fuzzing microservices in industry: Experience of applying evomaster at meituan. *arXiv preprint arXiv:2208.03988* (2022).
- [52] Man Zhang, Andrea Arcuri, Piyun Teng, Kaiming Xue, and Wenhao Wang. 2024. Seeding and Mocking in White-Box Fuzzing Enterprise RPC APIs: An Industrial Case Study. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento, CA, USA) (ASE '24)*. Association for Computing Machinery, New York, NY, USA, 2024–2034. <https://doi.org/10.1145/3691620.3695265>
- [53] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223* (2023).